MLOps Project Proposal

Intelligent Semantic Note Linking & Knowledge Discovery for Logseq

---

Open-Source Project: Logseq (32k+ GitHub Stars)

Hosted on: Chameleon Cloud

February 2026

1. Public-Facing Service Description

(Joint Responsibility — 0.5 points)

1.1 Service Overview

We propose deploying Logseq as a self-hosted, collaborative knowledge management platform on Chameleon Cloud. Logseq is a privacy-first, open-source note-taking and knowledge management application that uses a graph-based, outliner-style interface. It supports Markdown and Org-mode formats, bidirectional linking between pages, and a visual knowledge graph. The project has over 32,000 GitHub stars, an active development community, and is licensed under AGPL-3.0.

Our deployment will repurpose Logseq as a shared research knowledge base for an academic or small-team setting, where multiple users maintain interconnected notes, literature reviews, project documentation, and meeting notes.

1.2 Target Audience

Primary users: Graduate students, research lab members, and small academic teams (anticipated 50–200 active users).

Context: These users regularly create structured notes with references to papers, concepts, people, and projects. They frequently need to rediscover older notes and find conceptual connections across their knowledge base. Currently, Logseq requires users to manually create all links between pages, which means many valuable connections go undiscovered.

User pain point: As a knowledge graph grows (hundreds to thousands of pages), users cannot remember everything they have written. Important conceptual relationships between notes remain hidden because users must explicitly create every link. This is the gap our ML feature addresses.

1.3 Why Logseq?

- 32,000+ GitHub stars, well above the 2.5k minimum requirement
- Local-first architecture with clear data formats (Markdown files) that are easy to parse and process for ML pipelines
- The graph-based structure provides natural training signals (existing user-created links serve as ground truth for relationship modeling)
- Active plugin ecosystem allows complementary feature integration without modifying the core application
- Self-hostable and designed for privacy, aligning well with Chameleon deployment requirements

2. Complementary ML Feature Design

(Joint Responsibility — 2 points)

2.1 Feature: Semantic Note Linking & Knowledge Discovery

The ML feature automatically suggests relevant connections between notes that the user has not explicitly linked. When a user creates or edits a note block, the system analyzes its semantic content and recommends related pages, blocks, or concepts from elsewhere in their knowledge graph. This surfaces forgotten knowledge and reveals non-obvious conceptual relationships.

2.2 Design Specification (per 1.5.5 Specifying the Design)

2.2.1 What Does the ML Feature Do?

The feature provides a ranked list of suggested links for any given note block. When a user is viewing or editing a page, a sidebar panel displays "Suggested Connections" showing the top-5 most semantically relevant pages or blocks from the rest of their graph that are not already linked. Each suggestion includes a relevance score and a short snippet showing why the connection is relevant.

2.2.2 Who/What Interacts With the Feature?

Input trigger: A user views or edits a note page in Logseq.

Consumer of output: The user sees suggestions in a sidebar. They can accept a suggestion (which creates a bidirectional link in the graph), dismiss it (marking it as not relevant), or ignore it.

Automated downstream: Accepted and dismissed suggestions are logged as labeled training pairs for model retraining.

2.2.3 How Will Users Provide Feedback?

Feedback is collected through natural user interaction:

- Accept: User clicks a suggested link to add it to their note. This creates a positive training pair (query block, suggested block) with label = relevant.
- Dismiss: User clicks an × button on a suggestion. This creates a negative training pair (query block, suggested block) with label = not relevant.
- Implicit negative: Suggestions that are displayed but neither accepted nor dismissed after multiple exposures are treated as weak negatives.

This feedback mechanism is fully integrated into the user workflow — users do not need to perform any extra labeling work.

2.2.4 How Will Feedback Be Used for Retraining?

Feedback is accumulated in a structured log with fields: (timestamp, user_id, query_block_text, suggested_block_text, action, query_block_embedding, suggested_block_embedding). On a scheduled basis (see Section 7: Freshness Requirements), the system:

1. Extracts all new feedback pairs since the last retraining cycle.

2. Constructs contrastive training triplets: (anchor, positive, negative) from accepted vs. dismissed pairs.
3. Fine-tunes the embedding model using a contrastive loss (e.g., triplet loss or InfoNCE loss) on these new examples.
4. Evaluates the updated model on a held-out validation set of recent feedback.
5. If validation metrics improve, promotes the new model to production and rebuilds the embedding index.

2.2.5 What Outcome Does the Model Output Translate To?

The model outputs a ranked list of (candidate_block_id, relevance_score) pairs. In the real system, this translates to:

- A sidebar panel in the Logseq UI showing "Suggested Connections" with clickable links.
- Each suggestion shows the target page name, a relevant snippet, and the confidence score.
- Clicking a suggestion inserts a bidirectional link ([[page_name]]) into the current block.
- Over time, the graph becomes richer and more interconnected, improving the user's ability to navigate and rediscover knowledge.

2.3 System Architecture Overview

The system consists of four main components:

- Embedding Service: A microservice that encodes note blocks into dense vector representations using the fine-tuned sentence transformer model.
- Index Service: Maintains an approximate nearest neighbor (ANN) index (using FAISS or Annoy) over all block embeddings in the knowledge graph. Updated incrementally as notes change.
- Suggestion API: Receives a query block from the Logseq frontend, calls the embedding service, queries the ANN index, filters out already-linked pages, and returns ranked suggestions.
- Feedback Collector: Logs user interactions (accept/dismiss) and stores them for the retraining pipeline.

3. External Dataset

(Joint Responsibility — 0.5 points)

3.1 Primary Dataset: Wikipedia Hyperlink Graph

What: The Wikipedia hyperlink dataset, which captures the full set of internal links between Wikipedia articles. Each data point is a (source_article, anchor_text, target_article) triple.

Who collected it: The Wikimedia Foundation maintains the raw data dumps. Processed versions are available from academic researchers (e.g., the Stanford SNAP group and DBpedia project).

How: Extracted programmatically from Wikipedia XML dumps by parsing internal [[wikilinks]]. The dumps are released monthly by the Wikimedia Foundation.

Why: Wikipedia editors manually create links between articles when they determine a meaningful conceptual connection exists. This mirrors exactly what Logseq users do when creating bidirectional links between notes.

3.2 Alignment with the Service

Wikipedia's internal linking structure is an ideal proxy for Logseq's knowledge graph linking for several reasons:

- Both systems use bidirectional/internal links to connect related concepts.
- Links in both systems are created by humans based on perceived semantic relevance.
- Wikipedia's scale (6M+ articles, 200M+ links in English alone) provides abundant training signal.
- The anchor text surrounding links provides natural context for learning what makes two pieces of content "related."

3.3 Example Data Points

Below are representative examples from the dataset:

Source Article	Anchor Text Context	Target Article
Machine Learning	"...a subset of artificial intelligence that focuses on..."	Artificial Intelligence
Transformer (ML)	"...uses a self-attention mechanism first described in..."	Attention (ML)
Gradient Descent	"...is an optimization algorithm used to minimize the loss function in..."	Loss Function
BERT (language model)	"...pre-trained on large corpora using masked language modeling, similar to..."	Language Model

3.4 Secondary Dataset: S2ORC (Semantic Scholar Open Research Corpus)

What: A large corpus of academic papers with citation links, abstracts, and full text. Contains 81M+ papers with structured citation graphs.

Who collected it: Allen Institute for AI (AI2), as part of the Semantic Scholar project.

How: Aggregated from publisher APIs, open-access repositories (e.g., arXiv, PubMed Central), and web crawling of academic papers. Citation links are extracted from reference sections.

Why it aligns: Citation links between academic papers represent expert-identified conceptual relationships, analogous to how researchers using Logseq would link related notes about papers and concepts. This dataset is especially aligned with our target audience of graduate students and researchers.

3.5 Data Lineage Summary

Dataset	Creator	Collection Method	License
Wikipedia Links	Wikimedia Foundation	Monthly XML dumps parsed for internal links	CC BY-SA 4.0
S2ORC	Allen Institute for AI	Publisher APIs, arXiv, PubMed scraping	Open (ODC-BY)

4. Model Specification

(Training Team Member — 2/5 points)

4.1 Model Type and Architecture (1 point)

We use a bi-encoder (two-tower) architecture built on a sentence transformer. Specifically:

Base model: all-MiniLM-L6-v2 (a distilled 6-layer MiniLM model, 22M parameters, from the sentence-transformers library). This model is small enough to run efficiently on CPU while producing high-quality 384-dimensional sentence embeddings.

Architecture: The bi-encoder consists of two weight-sharing transformer encoders. One encodes the query block (the note being viewed/edited) and the other encodes candidate blocks. Both produce 384-dimensional normalized embeddings. Relevance is computed as the cosine similarity between the two embeddings.

4.1.1 Training Strategy

Phase 1 — Pre-training on Wikipedia: Fine-tune the base model on Wikipedia link pairs. For each (source_article, anchor_context, target_article), extract the paragraph containing the anchor text as the query and the first paragraph of the target article as the positive. Random articles serve as negatives. Training uses Multiple Negatives Ranking Loss (MNRL) with in-batch negatives, which is highly efficient.

Phase 2 — Domain adaptation on S2ORC: Further fine-tune on academic citation pairs (citing paper abstract → cited paper abstract) to adapt the model to academic writing style. Same loss function.

Phase 3 — Continual retraining on user feedback: Periodically fine-tune on accumulated (accepted_link, dismissed_link) pairs from production. Accepted pairs are positives; dismissed pairs are hard negatives. This phase uses triplet loss with user feedback triplets.

4.1.2 How the Model Will Be Retrained

Retraining follows a warm-start approach:

- The current production model weights are loaded as the starting point.
- New user feedback data is combined with a subset of the original training data (10% sampling) to prevent catastrophic forgetting.
- Training runs for 1–3 epochs on the combined dataset with a low learning rate (2e-5).
- The retrained model is evaluated on a held-out set of recent user feedback. If recall@5 improves by at least 1%, the new model is promoted.
- The ANN index is rebuilt using the new embeddings.

4.2 Input Features and Output (1 point)

4.2.1 Input Features

Each note block is processed into the following input representation:

- Text content: The raw Markdown text of the note block, with Logseq-specific syntax (e.g., [[page references]], #tags, TODO markers) converted to plain text tokens. Page references are expanded to include the referenced page title.
- Context window: The text of the parent block and up to 2 sibling blocks, providing local context (concatenated with [SEP] tokens).
- Page metadata: The page title and any page-level properties (e.g., tags, aliases) are prepended to the block text.

The combined input is tokenized using the MiniLM tokenizer and truncated to 256 tokens.

4.2.2 Output

Model output: A 384-dimensional normalized embedding vector for the input block.

System output: A ranked list of the top-K (default K=5) candidate blocks from the knowledge graph, sorted by cosine similarity to the query embedding. Each result includes:

- candidate_block_id: Unique identifier of the suggested block.
- candidate_page_title: The page containing the suggested block.
- relevance_score: Cosine similarity in [0, 1] between query and candidate embeddings.
- snippet: The first 100 characters of the candidate block for preview.

5. Serving & Operational Requirements

(Serving Team Member — 2/5 points)

5.1 Operational Requirements (1 point)

5.1.1 Throughput

Expected requests per second: 5–20 RPS during peak hours. With 50–200 active users, not all will be simultaneously editing notes. We estimate ~10% concurrent active editing, each generating a suggestion request on page focus or after a typing pause (debounced at 2 seconds).

Justification: 200 users × 10% concurrent × 1 request every 10 seconds = ~2 RPS typical, with spikes up to 20 RPS during busy periods (e.g., class or meeting times).

5.1.2 Latency

Target end-to-end latency: ≤ 200 ms (p95).

Latency budget breakdown:

Component	Target (ms)	Justification
Text preprocessing	5–10	Regex + tokenization
Embedding inference (MiniLM-L6)	30–60	CPU inference, 256 tokens
ANN index lookup (FAISS)	1–5	IVF index, top-50 retrieval
Filtering + ranking	5–10	Remove linked, rank top-5
Network overhead	10–20	Internal network
Total	51–105	Well within 200ms target

5.1.3 Availability and Error Handling

The suggestion feature is non-critical — if the ML service is down, Logseq continues to function normally without suggestions. We target 99% uptime. Errors return an empty suggestion list gracefully.

5.2 Model Output to Outcome Mapping (1 point)

The translation from model output to user-facing outcome follows this pipeline:

6. **Trigger:** User navigates to a page or pauses typing for 2+ seconds.
7. **Query construction:** The current block text + context is sent to the Suggestion API.
8. **Embedding:** The Embedding Service encodes the query into a 384-d vector.
9. **Retrieval:** FAISS index returns top-50 nearest neighbors.

10. **Post-processing:** Filter out blocks from the same page and already-linked pages. Re-rank remaining candidates. Select top-5.
11. **Presentation:** Display in Logseq sidebar with page title, snippet, and relevance indicator (high/medium/low, based on score thresholds).
12. **User action:** Accept → bidirectional link created; Dismiss → suggestion removed; Ignore → no action. All logged.

Relevance score thresholds for display:

- Score ≥ 0.85: Displayed as "Highly Relevant" with a green indicator.
- Score 0.70–0.85: Displayed as "Related" with a yellow indicator.
- Score 0.55–0.70: Displayed as "Possibly Related" with a gray indicator.
- Score < 0.55: Not displayed to the user.

6. Data Flow & Training Data

(Data Team Member — 2/5 points)

6.1 Data Flow Description (1 point)

6.1.1 Inference-Time Data Flow

When a user views or edits a note, the following data flow occurs:

13. **Data arrives:** The Logseq frontend sends the current block text, parent block text, sibling block texts, page title, and page tags to the Suggestion API as a JSON payload.
14. **Preprocessing:** The API service strips Logseq-specific Markdown syntax (e.g., converts [[Page Name]] to "Page Name", removes TODO/DONE markers, strips block-level properties). Concatenates page title + block text + context with [SEP] delimiters. Tokenizes using the MiniLM tokenizer and truncates to 256 tokens.
15. **Inference:** The preprocessed token sequence is passed through the MiniLM encoder to produce a 384-dimensional embedding vector. This is L2-normalized.
16. **Retrieval:** The normalized embedding is used to query the FAISS IVF (Inverted File) index, retrieving the top-50 nearest neighbors by inner product similarity.
17. **Filtering:** Results are filtered to remove same-page blocks, blocks from already-linked pages, and previously dismissed suggestions. Remaining candidates are re-ranked and the top-5 are returned.

6.1.2 Training-Time Data Flow

The training data pipeline processes data in batches:

18. **External data ingestion:** Wikipedia dump files are downloaded and parsed. For each internal link, the paragraph containing the link anchor is extracted as the source context, and the first paragraph of the target article is extracted as the target. These form (query, positive) pairs.
19. **Negative sampling:** For each positive pair, hard negatives are mined using BM25 retrieval (articles that are textually similar but not linked). Additional random negatives are sampled from the corpus.
20. **Preprocessing:** All text snippets undergo the same preprocessing as inference time: syntax stripping, tokenization, truncation to 256 tokens.
21. **Batch formation:** Training examples are grouped into batches of 64. Each batch is structured for Multiple Negatives Ranking Loss: positives are paired with in-batch negatives plus 1 hard negative per pair.
22. **User feedback integration:** During retraining, user feedback pairs are formatted as contrastive triplets: (query_block, accepted_block, dismissed_block). These are mixed with 10% of the original Wikipedia training data.

6.2 Training Data: Candidate Selection & Leakage Prevention (1 point)

6.2.1 Candidate Selection Strategy

We employ a multi-stage candidate selection approach:

Stage 1 — Corpus filtering: From the full Wikipedia dump (~6M articles in English), we filter to articles with at least 5 outgoing internal links and at least 200 words of body text, yielding approximately 3M qualifying articles.

Stage 2 — Link pair extraction: For each qualifying article, extract all internal links with their surrounding context (the sentence containing the link and one sentence before/after). Each link yields one training pair. Expected yield: ~50M raw training pairs.

Stage 3 — Quality filtering: Remove pairs where the source or target text is shorter than 20 tokens (too little semantic content). Remove redirect-only articles. Remove pairs where source and target are in the same disambiguation family. Final dataset: ~30M high-quality training pairs.

Stage 4 — Sampling for tractability: Randomly sample 2M pairs for initial training (stratified by Wikipedia category to maintain topic diversity). The full 30M set is available for continued training.

6.2.2 Avoiding Data Leakage

Data leakage is addressed at multiple levels:

Training/test split by time: For user feedback data, we split strictly by time. Feedback from the first N weeks is used for training; the most recent week is held out for evaluation. This prevents the model from seeing future user behavior during training.

No user content in external data: The Wikipedia and S2ORC datasets contain no user-generated Logseq content, so there is no direct leakage between the external training set and the user's knowledge graph.

Cross-user isolation: In a multi-user deployment, each user's feedback is kept separate. We do not train on User A's feedback and evaluate on User A's held-out data from the same time period. Evaluation uses time-based splits within each user.

No target leakage in features: The input features (block text, context, page title) do not include information about whether a link exists between the query and candidate blocks. The existence of links is the label, not a feature.

Embedding index isolation: During evaluation, the ANN index used for retrieval does not include any blocks from the evaluation set queries. Queries are encoded and searched against only blocks not in the evaluation batch.

7. DevOps & Platform Requirements

(DevOps/Platform Team Member — 2/5 points)

7.1 Freshness Requirements for Models (1 point)

7.1.1 Retraining Frequency

Scheduled retraining: Weekly, on Sunday nights during low-usage periods.

Triggered retraining: When accumulated new feedback exceeds 500 labeled pairs since the last retraining cycle, an immediate retraining is triggered (capped at once per 48 hours to prevent excessive resource consumption).

7.1.2 Justification

Weekly retraining is appropriate for this use case because:

- Note-linking preferences evolve slowly. Users' conceptual frameworks don't change dramatically day to day.
- The base model (pre-trained on Wikipedia) already provides strong general-purpose semantic similarity. User feedback fine-tuning provides incremental improvements.
- Weekly retraining balances model freshness against compute costs on Chameleon. Each retraining run takes approximately 30–60 minutes on a single GPU or 2–4 hours on CPU.
- The embedding index (FAISS) is rebuilt daily or when the number of new/modified notes exceeds 100, since this is fast (~5 minutes for 100K blocks) and ensures newly created notes are immediately discoverable.

7.1.3 Automation Lifecycle

The retraining pipeline fits into the following automation lifecycle:

23. **Data Collection (continuous):** Feedback collector logs user interactions to a PostgreSQL database.
24. **Data Validation (pre-training):** Automated checks verify minimum data volume, label distribution (not >90% positive or negative), and data quality (no empty texts).
25. **Training (weekly/triggered):** A cron job (or threshold-triggered webhook) launches the retraining script. Model artifacts are versioned and stored.
26. **Evaluation (automated):** Recall@5 and MRR (Mean Reciprocal Rank) are computed on the held-out validation set. Results are logged to a metrics dashboard.
27. **Promotion (gated):** If metrics improve, the new model is promoted. If not, the current production model is retained and the team is alerted.
28. **Index Rebuild (post-promotion):** After model promotion, all block embeddings are recomputed and the FAISS index is rebuilt. During rebuild, the old index continues serving.

7.2 Scaling Requirements (1 point)

7.2.1 Usage Estimates

Metric	Typical	Peak
Concurrent users	10–30	80–150
Requests per second	2–5 RPS	15–20 RPS
Knowledge graph size	10K–50K blocks	200K blocks
FAISS index memory	~50 MB	~300 MB
Model memory (MiniLM)	~90 MB	~90 MB

7.2.2 Right-Sizing Strategy

Minimum deployment (typical load): A single Chameleon m1.medium instance (4 vCPUs, 8 GB RAM) runs the Logseq web app, the suggestion API, the embedding service, and the FAISS index. This comfortably handles 5 RPS with sub-200ms latency. Total memory footprint: ~2 GB (model + index + application).

Scaled deployment (peak load): For peak periods, we horizontally scale the embedding/suggestion service to 2–3 replicas behind a load balancer (e.g., Nginx). Each replica loads its own copy of the model and index. Chameleon's reservation system supports pre-allocating instances during known peak hours.

Retraining resources: Retraining is run as a batch job on a separate Chameleon instance (ideally with GPU for faster training, but CPU-only is feasible). The training instance is reserved only during retraining windows and released afterward to conserve resources.

7.2.3 Resource Monitoring

We monitor the following metrics to guide scaling decisions:

- Request latency (p50, p95, p99) — alert if p95 > 200ms.
- CPU utilization — scale out if sustained > 70% for 10 minutes.
- Memory usage — alert if > 80% of available RAM.
- FAISS index size — if approaching memory limits, switch from flat index to IVF with product quantization.
- Suggestion acceptance rate — track as a proxy for model quality. Alert if drops below 10% (suggesting model degradation).